```
from collections import deque
import time

# Graph representing routes between locations
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['G'],
    'F': ['G'],
    'G': []
}

start = 'A'
goal = 'G'



# --------------------------------
# Breadth First Search (BFS)
# --------------------------------
def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = set()
    nodes_explored = 0

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node in visited:
```

```
            continue

        visited.add(node)
        nodes_explored += 1

        if node == goal:
            return path, nodes_explored

        for neighbour in graph[node]:
            if neighbour not in visited:
                new_path = path + [neighbour]
                queue.append(new_path)

    return None, nodes_explored



# --------------------------------
# Depth First Search (DFS)
# --------------------------------
def dfs(graph, start, goal):
    stack = [[start]]
    visited = set()
    nodes_explored = 0

    while stack:
        path = stack.pop()
        node = path[-1]

        if node in visited:
            continue
```

```
        visited.add(node)
        nodes_explored += 1

        if node == goal:
            return path, nodes_explored

        # Reverse order for predictable DFS traversal
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                new_path = path + [neighbour]
                stack.append(new_path)

    return None, nodes_explored



# ------------------------------
# Execute BFS
# ------------------------------
start_time = time.perf_counter()

bfs_path, bfs_nodes = bfs(graph, start, goal)

bfs_time = time.perf_counter() - start_time



# ------------------------------
# Execute DFS
# ------------------------------
start_time = time.perf_counter()

dfs_path, dfs_nodes = dfs(graph, start, goal)
```

```
dfs_time = time.perf_counter() - start_time



# ------------------------------
# Display Results
# ------------------------------
print("========== BFS RESULTS ==========")
print("Path:", " -> ".join(bfs_path))
print("Path Length:", len(bfs_path) - 1)
print("Nodes Explored:", bfs_nodes)
print("Execution Time: {:.8f} seconds".format(bfs_time))

print("\n========== DFS RESULTS ==========")
print("Path:", " -> ".join(dfs_path))
print("Path Length:", len(dfs_path) - 1)
print("Nodes Explored:", dfs_nodes)
print("Execution Time: {:.8f} seconds".format(dfs_time))



# ------------------------------
# Comparison
# ------------------------------
print("\n========== COMPARISON ==========")

if len(bfs_path) - 1 < len(dfs_path) - 1:
    print("BFS found the shorter path.")
elif len(dfs_path) - 1 < len(bfs_path) - 1:
    print("DFS found the shorter path.")
else:
    print("Both algorithms found paths of equal length.")
```

```
print("BFS Nodes Explored :", bfs_nodes)
print("DFS Nodes Explored :", dfs_nodes)

print("BFS Execution Time : {:.8f} seconds".format(bfs_time))
print("DFS Execution Time : {:.8f} seconds".format(dfs_time))
```